

CampusConnect

A University Marketplace, Jobs & Events Platform

Professional Project Report

University of Buea — Faculty of Engineering and Technology
Software Engineering & Digital Marketing

Table of Contents

1. Executive Summary

CampusConnect is a university platform that lets students buy and sell used items, advertise services, discover internships and jobs, browse campus events, and communicate with one another. The project is a cross-functional collaboration between the Software Engineering and Digital Marketing teams, pairing a working prototype with a go-to-market plan.

How to expand this section

Expand this section with the project's objectives, implementation approach, deliverables, team roles, key risks, acceptance criteria, and any lessons or best practices worth carrying forward. Support the narrative with diagrams, screenshots, and meeting notes gathered during the project.

2. Project Objectives

The team's objective is to deliver a functional prototype that also demonstrates planning, teamwork, communication, leadership, problem-solving, initiative, documentation, testing, and presentation skills.

How to expand this section

Expand this section with specific, measurable objectives tied to each deliverable, and note how each objective will be evidenced (e.g., a demo, a test report, a retrospective).

3. Stakeholders

Key stakeholders include the client, university students, student entrepreneurs, recruiters, event organizers, system administrators, the marketing team, and the development team.

How to expand this section

Expand this section with a stakeholder map: each group's interest in the project, their level of influence, and how the team plans to communicate with them.

4. Functional Requirements

Core functionality includes user registration and login, profile management, the marketplace, search and filtering, product detail pages, messaging, internship listings, campus events, admin moderation, and a responsive design across devices.

How to expand this section

Expand this section with a full requirements table (ID, description, priority, linked user story) and acceptance criteria for each feature.

5. Non-Functional Requirements

Non-functional priorities include performance, scalability, usability, accessibility, security, maintainability, reliability, availability, and responsiveness.

How to expand this section

Expand this section with measurable targets for each attribute — for example, target page-load time, concurrent-user capacity, or accessibility conformance level.

6. Software Architecture

The system is organized into a presentation layer (React), an API layer (Node.js/Express), a data layer (Firebase or PostgreSQL), an authentication layer, hosting, and a CI/CD pipeline.

How to expand this section

Expand this section with an architecture diagram, a description of how the layers communicate, and the rationale behind each technology choice.

7. UI/UX

Design work covers core design principles, wireframes, accessibility, typography, the color system, navigation patterns, and responsive layouts.

How to expand this section

Expand this section with wireframes or mockups, the design rationale, and notes from any usability testing conducted with real students.

8. Database Design

Core entities include Users, Listings, Categories, Services, Jobs, Events, Messages, Favorites, Reviews, and Notifications.

How to expand this section

Expand this section with an entity-relationship diagram, field-level schema definitions, and notes on indexing or normalization decisions.

9. API Design

The API surface includes authentication endpoints, CRUD operations, search APIs, messaging APIs, event APIs, and internship APIs.

How to expand this section

Expand this section with an endpoint reference (method, path, request/response schema, auth requirements) and versioning conventions.

10. Testing

The test plan spans unit, integration, system, usability, acceptance, regression, performance, and security testing.

How to expand this section

Expand this section with the test strategy per layer, coverage targets, tooling used, and a summary of defects found and resolved.

11. Deployment

Deployment relies on GitHub for source control, Vercel for the front end, and Render or Railway for backend hosting, with environment variables managed per environment and a production checklist before release.

How to expand this section

Expand this section with the deployment pipeline diagram, environment configuration details, and the production readiness checklist used.

12. Documentation

Supporting documentation includes the Software Requirements Specification (SRS), UML diagrams, an ER diagram, an installation guide, a user manual, and API documentation.

How to expand this section

Expand this section with links to each document and a short note on how documentation is kept in sync with the codebase.

13. Branding

Brand assets include the logo concept, color palette, typography, tone of voice, brand guidelines, and tagline.

How to expand this section

Expand this section with the finalized brand guidelines and examples of the brand applied across the product and marketing materials.

14. Market Research

Research covers competitor analysis, market opportunities, the target audience, user personas, and a SWOT analysis.

How to expand this section

Expand this section with the competitor comparison table, persona profiles, and the full SWOT findings.

15. Marketing Strategy

The go-to-market plan includes a social media content calendar, a launch campaign, a growth strategy, and a customer acquisition plan.

How to expand this section

Expand this section with the campaign timeline, channel mix, and the metrics used to evaluate early traction.

16. Business Model

Potential revenue streams include partnerships, premium listings, sponsorships, advertising, and other future monetization options.

How to expand this section

Expand this section with a revenue model breakdown and the assumptions behind each stream.

17. Agile Management

The team follows an Agile process built around sprint planning, a maintained backlog, daily stand-ups, retrospectives, and issue tracking.

How to expand this section

Expand this section with the sprint cadence, backlog snapshots, and key takeaways from retrospectives.

18. Risk Management

Identified risk categories include technical risk, communication risk, and schedule risk, each with a corresponding mitigation plan.

How to expand this section

Expand this section with a risk register (likelihood, impact, owner, mitigation) tracked throughout the project.

19. Presentation Plan

The final presentation is structured around an introduction, a live demo, the architecture, marketing, business strategy, challenges faced, lessons learned, and future work.

How to expand this section

Expand this section with the presentation outline, time allocation per segment, and presenter assignments.

20. Future Enhancements

Planned enhancements include AI-driven recommendations, a mobile application, push notifications, real-time chat, analytics, and payment integration.

How to expand this section

Expand this section with a prioritized roadmap and the rationale for the proposed sequencing.

21. Appendices

The appendices hold the supporting evidence for the report above. Rather than reserving 30 near-identical placeholder pages, use the checklist below to organize evidence into a small number of clearly labeled appendices as the project produces them.

Suggested appendix structure

- Appendix A — Meeting minutes and sprint backlog
- Appendix B — Git commit history and task allocation
- Appendix C — UML and ER diagrams
- Appendix D — Wireframes and UI mockups
- Appendix E — API specifications
- Appendix F — Test cases and results
- Appendix G — Screenshots and demo evidence
- Appendix H — Marketing assets and survey results
- Appendix I — Branding materials
- Appendix J — Presentation notes, installation steps, and deployment logs

Replace each placeholder with the team's actual work product before final submission, and remove any appendix category that does not apply.